Budget and Expense Analysis Tool Implementation Plan

Goal: Build a polished, production-deployed budgeting web application that lets users import, categorize, analyze, and visualize expenses while demonstrating strong frontend, backend, data, testing, and deployment skills in a portfolio.

Architecture: Use a monorepo containing a React/Vite/TypeScript single-page application and a Hono API deployed to Cloudflare Workers. Store application data in Cloudflare D1 (SQLite) through Drizzle ORM. Deploy the frontend to Cloudflare Pages and connect the user's Cloudflare-managed domain with a custom DNS record. The first release should support a demo mode without authentication, then add per-user accounts only after the core product is stable.

Tech Stack: TypeScript, React, Vite, Tailwind CSS, shadcn/ui, Recharts, TanStack Query, React Hook Form, Zod, Hono, Cloudflare Workers, D1, Drizzle ORM, Vitest, Testing Library, Playwright, ESLint, Prettier, GitHub Actions, Cloudflare Pages/Workers, Wrangler.

1. Product definition and portfolio scope

Primary user journey

1. Open the public landing page and start the sample dashboard.
2. Import a CSV transaction file or enter transactions manually.
3. Review and correct transaction categories.
4. Set monthly income and category budgets.
5. View spending, budget progress, trends, recurring expenses, and savings insights.
6. Filter by date, account, category, and transaction type.
7. Export a filtered report as CSV.
8. On a later iteration, create an account so data persists privately across devices.

MVP features

- Responsive dashboard with current-month summary cards.
- Transactions table with search, sorting, pagination, filters, edit, delete, and category assignment.
- CSV import with column mapping, validation errors, duplicate detection, and preview before saving.
- Categories with sensible defaults and user-created categories.
- Monthly budget setup by category.
- Charts: spending by category, monthly trend, budget-vs-actual, and income-vs-expense.
- Demo dataset and a clearly marked demo mode.
- CSV export.
- Empty, loading, error, and mobile states.
- Accessibility basics: keyboard navigation, visible focus, labels, sufficient contrast, and chart data available in text.

Deliberately defer

- Bank account synchronization and financial institution credentials.
- Investment, tax, debt, or financial-advice features.
- Complex forecasting or machine-learning recommendations.
- Native mobile applications.
- Multi-currency conversion unless it is explicitly required.
- Paid subscriptions.

These deferred features increase security, compliance, and maintenance scope without improving the first portfolio release proportionally.

2. Repository and tool setup

Target repository structure

```
.
■■■ apps/
■   ■■■ web/                # React/Vite frontend
■   ■■■ api/                # Hono Cloudflare Worker API
■■■ packages/
■   ■■■ shared/             # Shared Zod schemas and TypeScript types
■■■ db/
■   ■■■ schema.ts           # Drizzle schema
■   ■■■ seed.ts             # Demo data seed
■■■ tests/
■   ■■■ e2e/                # Playwright journeys
■   ■■■ fixtures/           # CSV fixtures and test data
■■■ .github/workflows/ci.yml
■■■ docs/                  # Architecture, data model, and deployment notes
■■■ package.json
■■■ pnpm-workspace.yaml
■■■ tsconfig.json
■■■ drizzle.config.ts
■■■ wrangler.toml
■■■ README.md
```

Tools and purpose

Tool	Use
VS Code	Primary editor and debugging workflow
Git/GitHub	Version control, portfolio history, pull requests
pnpm	Fast, reproducible monorepo dependency management
Node.js LTS + TypeScript	Runtime and application language
Vite	Fast frontend development/build tooling
React + Tailwind + shadcn/ui	Accessible, polished UI foundation
Recharts	Dashboard visualizations
TanStack Query	API caching, loading, and mutation state
Hono	Lightweight API routing on Workers
Zod	Shared runtime validation for forms, CSV rows, and API payloads
Drizzle ORM	Type-safe D1 schema and migrations
Cloudflare D1	Relational persistence close to the Worker

Cloudflare Pages	Frontend hosting and preview deployments
Cloudflare Workers	API hosting
Wrangler	Local D1/Worker development and deployment
Vitest + Testing Library	Unit and component tests
Playwright	Browser-level acceptance tests
GitHub Actions	CI for lint, typecheck, tests, and build
Tool	Use
Lighthouse/PageSpeed	Performance and accessibility checks
Cloudflare DNS	Custom-domain DNS and HTTPS routing

Setup tasks

1. Initialize the pnpm workspace and TypeScript configuration.
2. Create the web, API, shared, database, docs, and test directories.
3. Add strict TypeScript, ESLint, Prettier, and import/order conventions.
4. Configure environment files with `.env.example`; never commit real secrets.
5. Add scripts for `dev`, `build`, `lint`, `typecheck`, `test`, `test:e2e`, `db:generate`, `db:migrate`, and `db:seed`.
6. Add a README with local setup, architecture, screenshots placeholder, and deployment overview.
7. Confirm the empty application starts locally before adding features.

3. Data model and business rules

Initial tables

Create these tables in `db/schema.ts`:

- `users`: only needed when authentication is introduced; include provider subject ID and timestamps.
- `accounts`: optional source account name/type and owner ID.
- `categories`: owner ID, name, kind (`income`, `expense`, `transfer`), color, and archived flag.
- `transactions`: owner/demo scope, account ID, date, description, amount in integer minor units, currency, category ID, transaction type, import fingerprint, notes, and timestamps.
- `budgets`: owner/demo scope, month represented as `YYYY-MM-01`, category ID, limit in minor units, and timestamps.
- `imports`: owner/demo scope, original filename, row count, accepted count, rejected count, and timestamps.

Business rules

- Store monetary values as integer minor units, never floating-point currency values.
- Use ISO dates and UTC timestamps at the API boundary.
- Define whether positive values mean income and negative values mean expenses; normalize imported files to one internalconvention.
- Reject malformed dates, missing descriptions, unsupported currencies, and non-numeric amounts with row-level errors.
- Deduplicate imports using a stable fingerprint of date, amount, description, and account/source.

- Treat transfers separately so they do not inflate income or expense totals.
- Compute summaries server-side or in a shared pure module so dashboard numbers and exports agree.
- Add indexes for owner/demo scope, transaction date, category, and import fingerprint.

Required tests before UI integration

- Amount normalization and minor-unit conversion.
- Date parsing and timezone behavior.
- Transaction classification.
- Duplicate fingerprinting.
- Monthly totals and budget percentage calculations.
- Empty data and over-budget behavior.

4. API implementation

API routes in `apps/api/src/routes/`

- `GET /health` — deployment/readiness check.
- `GET /api/dashboard?from=&to=` — summary cards and chart series.
- `GET /api/transactions` — paginated/filterable transaction list.
- `POST /api/transactions` — create one transaction.
- `PATCH /api/transactions/:id` — edit a transaction.
- `DELETE /api/transactions/:id` — delete a transaction.
- `POST /api/imports/preview` — parse and validate a CSV without persisting it.
- `POST /api/imports/commit` — persist an approved import.
- `GET /api/categories` and `POST /api/categories` — list/create categories.
- `PATCH /api/categories/:id` — rename/archive a category.
- `GET /api/budgets?month=` and `PUT /api/budgets` — read/upsert monthly budgets.
- `GET /api/export/transactions` — return a filtered CSV download.

API implementation order

1. Add Hono app bootstrap and centralized error handling.
2. Add shared request/response schemas in `packages/shared/src/schemas.ts`.
3. Add database access functions in `apps/api/src/db/` rather than embedding SQL in route handlers.
4. Implement read-only health and dashboard endpoints.
5. Implement transaction CRUD with schema validation and ownership/demo-scope checks.
6. Implement CSV preview and commit as two separate operations.
7. Implement categories, budgets, and export.
8. Add rate limits or basic abuse protection to write/import routes before production deployment.
9. Add API tests for valid input, invalid input, not-found behavior, duplicate imports, and scope isolation.

API security requirements

- Validate every request with Zod.

- Use parameterized Drizzle queries.
- Enforce maximum CSV size and maximum row count.
- Do not log raw transaction descriptions or personal financial data.
- Return generic server errors to clients and detailed errors only to server logs.
- Add CORS restricted to the production frontend and local development origins.
- If authentication is added, derive owner scope from the verified session, never from a client-provided user ID.

5. Frontend implementation

Pages and components

Create the following in `apps/web/src/`:

- `pages/LandingPage.tsx` — product explanation and demo CTA.
- `pages/DashboardPage.tsx` — summary, charts, filters, and insights.
- `pages/TransactionsPage.tsx` — table and transaction actions.
- `pages/ImportPage.tsx` — CSV upload, mapping, preview, errors, and commit.
- `pages/BudgetsPage.tsx` — monthly budget editor and progress.
- `components/layout/AppShell.tsx` — responsive navigation and page frame.
- `components/dashboard/MetricCard.tsx`, `SpendingByCategory.tsx`, `MonthlyTrend.tsx`, `BudgetProgress.tsx`.
- `components/transactions/TransactionTable.tsx`, `TransactionForm.tsx`, `TransactionFilters.tsx`.
- `components/import/ColumnMapper.tsx`, `ImportPreview.tsx`, `ImportErrors.tsx`.
- `lib/api.ts`, `lib/formatters.ts`, `lib/csv.ts`, and `lib/queryKeys.ts`.

Frontend implementation order

1. Establish design tokens, typography, color semantics, responsive breakpoints, and dark/light behavior.
2. Build the app shell and route structure with mock data.
3. Add query hooks and loading/error states.
4. Replace mock dashboard data with the API.
5. Add transaction table and CRUD forms.
6. Add CSV import wizard with a preview-first flow.
7. Add budgets and charts.
8. Add export and shareable demo behavior.
9. Add accessible keyboard and screen-reader behavior.
10. Add mobile layout testing at narrow viewport widths.

UX and visual quality bar

- Use plain-language labels such as “Money in,” “Money out,” and “Remaining budget.”
- Always show the date range behind totals.
- Avoid red/green-only meaning; pair color with labels/icons.
- Make import errors actionable by row and column.
- Include a small “How this is calculated” explanation for key metrics.

- Add realistic seeded sample data so the dashboard looks credible in a portfolio screenshot.
- Include a privacy note: “Demo data only; do not upload sensitive financial information.”

6. Authentication and privacy phase

Do not block the MVP on authentication. After the demo workflow is complete:

1. Choose one identity provider after comparing Cloudflare compatibility, cost, and portfolio value. A hosted provider such asClerk/Auth0 is simpler; a Worker-compatible session implementation gives more ownership but more security responsibility.
2. Add `users` and ownership columns to all user-owned tables.
3. Add login/logout and protected routes.
4. Migrate demo records to an explicit demo tenant rather than treating demo mode as a real user.
5. Add account deletion and data export for user data.
6. Document data retention, privacy limitations, and that the app is not a bank connection or financial-advice service.
7. Test cross-user isolation before enabling persistent accounts publicly.

7. Testing and quality gates

Test layers

- Unit tests: `packages/shared/tests/` for calculations, parsing, and validation.
- API tests: `apps/api/tests/` for route behavior and D1 integration.
- Component tests: `apps/web/src/**/*.test.tsx` for forms, filters, error states, and accessible labels.
- E2E tests: `tests/e2e/` for the main user journeys.

Required E2E journeys

1. Landing page opens demo dashboard.
2. Dashboard totals match seeded transactions.
3. User filters transactions and exports the filtered result.
4. User previews a valid CSV and sees invalid-row errors.
5. User commits a valid import and sees totals update.
6. User changes a category and sees category chart values update.
7. User sets a budget and sees budget progress update.
8. Mobile viewport remains usable.

CI gates

GitHub Actions should run on pull requests and the main branch:

```
pnpm install --frozen-lockfile
pnpm lint pnpm typecheck pnpm
test -- --run pnpm build pnpm
test:e2e
```

Add Lighthouse checks for performance, accessibility, best practices, and SEO. Set an initial target of 90+ for accessibility and best practices and improve performance after measuring rather than prematurely optimizing.

8. Cloudflare deployment and custom DNS

Cloudflare resources

Create separate resources for preview/staging and production where practical:

- Cloudflare Pages project for `apps/web`.
- Cloudflare Worker for `apps/api`.
- Cloudflare D1 database for production and a separate local/staging database.
- Optional Worker custom domain or Pages Functions only if it simplifies routing; avoid duplicating API implementations.

Deployment sequence

1. Create the GitHub repository and push the application.
2. Create the D1 databases with Wrangler and apply migrations.
3. Seed only demo data into the demo/staging database; never seed fabricated personal data into a user account.
4. Deploy the Worker API and verify `GET /health`.
5. Configure Pages to build from the web workspace with the exact pnpm install/build commands.
6. Configure production environment variables and bindings in Cloudflare, not in Git.
7. Connect the Cloudflare-managed domain in Pages or Workers according to the chosen routing design.
8. Add the DNS record Cloudflare requests (typically a proxied CNAME for a subdomain, or the apex setup managed byCloudflare).
9. Verify automatic TLS, the canonical URL, redirects, API CORS, and no mixed-content warnings.
10. Add a post-deployment smoke test against the production health endpoint and landing page.

Suggested domain layout

- `budget.example.com` — public web application.
- `api.budget.example.com` — Worker API, only if separate API routing is necessary.
- `www.example.com` — redirect to the canonical app URL.

Do not change DNS until the production build has passed local and preview verification. Record the final DNS/routing design in `docs/deployment.md`.

DNS and deployment verification checklist

- `dig`/DNS lookup resolves the intended hostname.
- HTTPS certificate is valid.
- `/health` returns a successful response.
- Demo dashboard loads with no console errors.
- CSV import and export work in production.
- API rejects requests from an unapproved origin.
- Cloudflare logs show no repeated 5xx errors.
- Rollback procedure is documented and tested with a previous deployment.

9. Portfolio presentation

Add these to `README.md` and `docs/`:

- One-paragraph problem statement and target user.

- Architecture diagram showing browser → Pages → Worker → D1.
- Screenshots or a short screen recording.
- Sample CSV format and demo instructions.
- Explanation of data normalization and calculation rules.
- Testing strategy and CI status.
- Performance/accessibility results.
- Security/privacy limitations.
- Live demo URL and source repository.
- “What I would build next” section covering bank integrations, authentication improvements, and forecasting.

The project should communicate engineering judgment, not just UI polish: show why D1 was selected, how duplicate imports are prevented, how money is represented safely, and how deployment is automated.

10. Suggested milestone sequence

Milestone 1 — Foundation

Workspace, linting, testing, shared schemas, database schema, seed data, local development, and README.

Milestone 2 — Core data workflow

Transaction CRUD, CSV preview/commit, category management, calculations, and API tests.

Milestone 3 — Usable product

Dashboard, charts, filters, budgets, export, responsive states, and component tests.

Milestone 4 — Production readiness

E2E tests, accessibility/performance checks, CI, Cloudflare preview deployment, production deployment, and custom domain DNS.

Milestone 5 — Portfolio refinement

Architecture documentation, screenshots, demo data quality, privacy copy, error polish, and a concise project case study.

Milestone 6 — Optional persistence

Authentication, ownership enforcement, data export/deletion, and production privacy review.

11. Risks, tradeoffs, and open decisions

- **Cloudflare-native simplicity vs. ecosystem familiarity:** React/Vite + Hono + D1 is a strong Cloudflare portfolio story, but a conventional Node/Postgres stack has more tutorials and libraries. Stay Cloudflare-native unless a required dependency is incompatible.
- **Authentication:** Adding it too early can delay the core product; adding it later requires ownership-aware schema design. Include nullable/tenant scope from the beginning so the migration is manageable.
- **CSV variability:** Real bank CSV files differ widely. Start with a documented template plus flexible column mapping; do not promise universal bank compatibility.
- **Financial accuracy:** Integer minor units, deterministic calculations, and comprehensive tests are mandatory. Never use JavaScript floating-point arithmetic for stored money.
- **Domain choice:** The exact domain and whether the app is hosted at the apex or a subdomain remain open. Use a subdomain by default to reduce risk to any existing website/email DNS records.
- **Cloudflare account access:** Deployment requires the user's Cloudflare account, zone, and API permissions. Keep account login and DNS changes as explicit deployment steps, not hidden automation.

Definition of done

- A visitor can open the custom domain and use the demo workflow end-to-end.
- Dashboard calculations match tested transaction fixtures.
- CSV import gives a preview, reports row-level errors, prevents duplicates, and updates the dashboard after commit.
- Budgets and charts work on desktop and mobile.
- Unit, API, component, E2E, lint, typecheck, and production build checks pass in CI.
- Cloudflare Pages/Workers/D1 deployment is reproducible from documented commands.
- Custom DNS, HTTPS, CORS, and rollback are verified.
- README and documentation make the project understandable to a recruiter or reviewer.